

**ТЕХНОЛОГИЧНО УЧИЛИЩЕ ЕЛЕКТРОННИ СИСТЕМИ
към ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ**



КУРСОВ ПРОЕКТ

Тема:
Състезателна бесеница
Клиент-Сървър игра по TCP

Ученик:
Калоян Янев

Дисциплина:
Операционни Системи

СОФИЯ
2026

1. Въведение

Проектът е съставен от три собствено разработени компонента — сървър (hangman-server), клиент (hangman-client) и помощна библиотека за работа с редове по сокети (net-lib) — както и от предоставената със заданието библиотека за игровата логика (game.c и game.h), която не е променяна.

Ролята на съдия се изпълнява от сървъра — единствено в него се съхраняват двете тайни думи, проверяват се предположенията, броят се грешките и се обявява победителят. Клиентът представлява интерфейс, чрез който състоянието се визуализира и натиснатите букви се препращат. Комуникацията се осъществява чрез текстов протокол, организиран по редове, върху TCP.

2. Условие на задачата (Вариант 3)

Играта се играе от двама участници. Всеки участник избира тайна дума, която трябва да бъде отгатната от неговия опонент. Двете думи се отгатват едновременно. За победител се определя участникът с по-малък брой грешни предположения; при равен брой грешки резултатът е равенство.

Със заданието се предоставя готова библиотека (game.c, game.h) с игровата логика, чието изменение не е разрешено. Поставят се следните изисквания:

- **Сървър** (hangman-server): приема един аргумент (порт); слуша винаги на 0.0.0.0; при грешка със сокет се извежда съобщение на стандартния изход за грешки и програмата приключва с код 1; при готовност се извежда точно „Listening on <порт>...“; приемат се точно двама клиенти, след което приемането се преустановява; невалидна дума се отхвърля със съобщение за грешка и връзката се затваря; думата за отгатване никога не се изпраща на клиента (защита срещу подказване).
- **Клиент** (hangman-client): стартира се с аргументи „<хост> <порт> <дума-за-опонента>“; в цикъл се извежда състоянието на думата, която се отгатва от участника, прочита се по една буква и се изпраща; накрая се извеждат резултатът и грешните букви на двамата участници.

3. Обща архитектура: клиент-сървър

Тъй като играта се води от двама участници едновременно, от съществено значение е мястото, на което се съхранява състоянието на играта. Ако при всеки клиент се пазеше отделно копие на думите и точките, между двете програми би могло да възникне несъответствие, а отговорът би бил достъпен в паметта на съответната програма. По тази причина се използва модела клиент-сървър.

3.1 Сървърът

Единственият достоверен източник на състояние се съхранява в сървъра. В него се пазят двете тайни думи, броят грешки за всеки участник и информацията кой е отгатнал думата си. Правилата на играта се прилагат единствено на това място.

3.2 Клиентът

Всеки участник разполага с клиент, в който нарочно не е вложена игрова логика — извършват се само визуализация на полученото състояние и препращане на написаните букви. Тъй като търсената дума никога не е достъпна за клиента, възможност за подсказване не съществува. Този модел съответства на принципа, по който функционират голяма част от мрежовите multiplayer игри - сървърът разполага с цялата информация, а клиентите просто могат да я получават или изпращат. Това се прави с цел сигурност, както и олекотяване на клиентската програма.

4. Обосновка на избора на TCP

Между двата процеса се обменят байтове по мрежата. От операционната система се предоставят два основни механизма за това — TCP и UDP. Изборът на TCP е направен съзнателно и има съществено значение.

4.1 Изисквания към транспорта

От транспортния протокол за нуждите на играта се изискват следните свойства:

- **Надеждност** — при загуба на предположена буква играта би била нарушена, поради което се изисква всяко съобщение да бъде доставено.
- **Подреденост** — предположенията и обновяванията на състоянието трябва да бъдат получавани в реда на изпращането им.
- **Връзка** — сокетът на единия участник трябва да бъде разграничаван от сокета на другия, за да бъдат поддържани две отделни състояния.

4.2 Сравнение между TCP и UDP

Протоколът UDP функционира по принципа „изпращане без потвърждение“: даден пакет може да бъде доставен, да бъде изгубен, дублиран или да бъде получен в различен ред, като връзка не се поддържа. Този протокол е подходящ за приложения като предаване на видео в реално време, при които закъснението е по-нежелателно от загубата. Ако за настоящата задача беше използван UDP, механизмите на TCP би трябвало да бъдат реализирани наново — потвърждения, повторно изпращане на изгубени данни, откриване на дубликати и подреждане — което представлява значителен и чувствителен към грешки обем код за проблем, който вече е решен от TCP. Също така ако липсваше подредбата на пакетите, предоставена от TCP, намирането и отстраняването на грешките в кода би било почти невъзможно. Тъй като

разбъркването на пакетите би било изцяло случайно, причината за това и поведението на играта биха били много странни и трудни за оправяне.

Чрез TCP се осигуряват наготово надеждна доставка (изгубените данни се откриват и изпращат повторно автоматично), доставка в реда на изпращане, дълготрайна двупосочна връзка между точно две страни (което съответства на отношението „един участник — сървър“) и контрол на потока. Единственият недостатък на TCP е неговата бавна скорост, както и по-сложната и натоварваща комуникация на самия протокол. Това обаче не е проблем в този случай, тъй като всеки играч изпраща по няколко букви и то в рамките на няколко секунди и дори минути. За това миниатюрно количество данни, изпратени в голям диапазон от време TCP изобщо не се задъхва и е абсолютно използваем.

5. Принцип на действие на TCP

В настоящия раздел се описва какво се извършва под извикванията `socket`, `connect`, `read` и `write`. Тези действия се изпълняват от ядрото на операционната система, а не от приложния код. Разбирането им обяснява устройството на помощната библиотека `net-lib`.

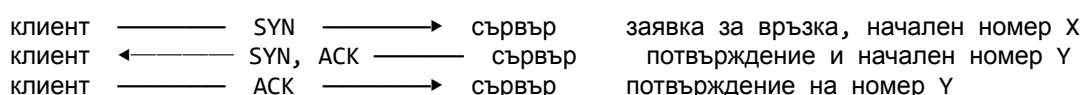
5.1 Сокети, адреси и портове

Сокетът представлява абстракция на операционната система за единия край на мрежова връзка. От страна на приложението той се използва чрез цяло число — файлов дескриптор — от което се чете и в което се записва аналогично на файл. Една TCP връзка се идентифицира еднозначно чрез адрес на изпращателя, порт на изпращателя, адрес на получателя и порт на получателя.

Машината се идентифицира чрез IP адреса (например 127.0.0.1 обозначава локалната машина), а конкретната програма на нея — чрез порта (16-битово число в интервала 0–65535). Благодарение на тази четворка от сървъра могат да бъдат поддържани две връзки от двама клиенти без смесване, дори когато клиентите се намират на една и съща машина, тъй като клиентският порт е различен.

5.2 Three-way handshake

Преди обмена на данни връзката се установява чрез размяна от три стъпки, която се задейства от `connect` при клиента и от `accept` при сървъра:



След размяната на тези три пакета началните поредни номера са съгласувани и връзката се счита за установена; едва тогава се допуска обмен на данни.

5.3 Поредни номера, потвърждения и повторно изпращане

Връзката се третира от TCP като непрекъснат поток от байтове, като всеки байт се номерира. При изпращане на данни от получателя се връща потвърждение (ACK), с което се обозначава, че всички байтове до определена позиция са приети. При липсата на потвърждение в рамките на определено време данните се считат за изгубени и се изпращат повторно. Пакети, получени в нарушен ред, се задържат и потокът се сглобява в правилната последователност, преди да бъде предаден на приложението. Дублираните пакети се разпознават по поредните им номера и се отстраняват. По този начин се гарантира, че всяка буква се доставя точно веднъж и в реда на изпращане, без да е необходимо реализиране на логика за повторно изпращане в приложния код.

5.4 Контрол на потока

Чрез контрола на потока всяка страна обявява наличния обем буфер, поради което от изпращателя не се изпраща повече, от колкото може да бъде прието от получателя. Чрез контрола на трафика, мрежата се наблюдава за признаци на претоварване и темпът на изпращане се намалява при необходимост. За настоящата задача тези механизми са без практическо значение, но са част от причините за надеждността на TCP в споделени мрежи.

5.5 TCP като поток от байтове

Това свойство е от съществено значение, тъй като с него се обуславя необходимостта от библиотеката net-lib. Границите на съобщенията не се запазват от TCP. При бързо изпращане на три букви от клиента

```
write("a\n")
write("b\n")
write("c\n")
```

едно извикване на read от страна на сървъра може да върне „a\nb\nc\n“ наведнъж или произволно друго разделение на тези байтове. От TCP се гарантира единствено, че байтовете се доставят надеждно и в ред, но не и начинът на групирането им в отделните извиквания на read. Следователно, когато протоколът е организиран по редове, откриването на границите на редовете (символа за нов ред) се извършва от приложението чрез буфериране на постъпващите байтове и разделянето им на редове.

5.6 Затваряне на връзката

При приключване от едната страна се изпраща пакет FIN, който се потвърждава от насрещната страна, след което и от нея се изпраща FIN. Извикване на read, което върне 0 байта, се интерпретира като затваряне на връзката от насрещната страна (край на потока). Именно тази стойност се проверява в кода с цел откриване на разкачен участник.

6. Жизнен цикъл на сокетите

6.1 От страна на сървъра

```
listen_fd = socket(AF_INET, SOCK_STREAM, 0);    // TCP/IPv4 сокет
setsockopt(... SO_REUSEADDR ...);              // преизползване на порта
bind(listen_fd, &addr ...);                    // заемане на порта на 0.0.0.0
listen(listen_fd, 8);                          // начало на приемането
printf("Listening on %d...\n", port);
cfd = accept(listen_fd, NULL, NULL);            // изчакване на клиент
```

Чрез аргументите `AF_INET` и `SOCK_STREAM` се указва използване на TCP върху IPv4. Сокетът се привързва към конкретен порт чрез `bind`, като чрез `INADDR_ANY` (0.0.0.0) се приемат връзки по всички мрежови интерфейси на машината, както се изисква от заданието. Чрез опцията `SO_REUSEADDR` се позволява повторно стартиране на сървъра непосредствено след спирането му. Чрез `listen` сокетът се превключва в пасивен режим. Чрез `accept` се изпълнява сървърската част от ръкостискането и се връща нов сокет, предназначен за съответния клиент, докато първоначалният сокет продължава да слуша, поради което от сървъра могат да бъдат поддържани два независими клиентски сокета.

6.2 От страна на клиента

```
getaddrinfo(host, port, &hints, &res);         // преобразуване на адреса
fd = socket(...);                             // TCP/IPv4 сокет
connect(fd, rp->ai_addr, rp->ai_addrlen);       // ръкостискане към сървъра
```

Чрез `getaddrinfo` името на хоста и портът се преобразуват в двоичната адресна структура, необходима за сокет-извикванията; при това може да бъде върнат списък от адреси, всеки от които се опитва последователно, докато `connect` приключи успешно. Чрез `connect` се изпълнява клиентската част от ръкостискането, след което връзката е установена.

7. Комуникационен протокол

Върху надеждния байтов поток на TCP се надгражда приложен протокол, чрез който се определя значението на обменяните байтове. В настоящата реализация този протокол е текстов и опростен.

7.1 Формат

Всяко съобщение представлява един ред текст, завършващ със символ за нов ред. Текстовият формат е предпочетен пред двоичен, тъй като чрез него се улесняват четенето, тестването и отстраняването на грешки, а обемът на данните е малък и темпът им съответства на въвеждане от човек, поради което компактността на двоичен формат не носи предимство.

7.2 Съобщения от клиента към сървъра

От клиента към сървъра се изпращат два вида съобщения. Веднага след свързването се изпраща един ред с думата, която ще трябва да бъде отгатната от опонента.

Впоследствие при всеки ход се изпраща ред с една предположена буква.

7.3 Съобщения от сървъра към клиента

От сървъра към клиента се изпращат три вида съобщения:

- ERR <текст> — обозначава невалидна дума; връзката се затваря след изпращането му.
- STATE <маска> <грешки> — представя текущия изглед на думата, която се отгатва от съответния участник.
- RESULT <код> <твои> <опоненти> — обозначава край на играта, като кодът приема стойност WIN, LOSE или TIE.

В полето „маска“ познатите букви се представят на местата им, а всяка все още скрита буква се обозначава със символа за подчертаване (например h_ll_), поради което цялата тайна дума никога не се изпраща — с което се реализира високото ниво на сигурност.

В полето „грешки“ грешните букви се изброяват, разделени със запетая, или се обозначава тире при липса на такива. От клиента този запис се преобразува за извеждане във вид с разделител запетая и интервал. В съобщението RESULT грешните букви на съответния участник и тези на опонента се предават поотделно, така че от клиента да бъдат изведени трите изисквани реда.

7.4 Кръстосано разпределение на думите

Думата на всеки участник се избира от опонента му, поради което третият аргумент при стартиране на клиента съдържа думата за опонента. На сървъра, първият свързал се участник се обозначава като players[0] и подава дума W0, а вторият — като players[1] и подава дума W1. Разпределението се извършва кръстосано:

```
secret_word_init_from_c_string(&players[0].target, words[1]); // P0 отгатва W1
secret_word_init_from_c_string(&players[1].target, words[0]); // P1 отгатва W0
```

Полето players[i].target обозначава думата, която трябва да бъде отгатната от участник i, и се задава на думата, подадена от другия участник. Чрез тази размяна се реализира изцяло правилото за избор на дума за опонента.

8. Библиотека net-lib

Необходимостта от тази помощна библиотека произтича от свойството, описано в раздел 5.5: байтовете се подават от TCP без граници на съобщенията, докато протоколът е организиран по редове. Сглобяването на редовете се осигурява от net-lib.

8.1 Структура `line_reader_t`

```
typedef struct line_reader {  
    int    fd;           // сокетът, от който се чете  
    char   buf[4096];    // получени, но още необработени байтове  
    size_t len;          // брой валидни байтове в buf  
} line_reader_t;
```

Структурата се състои от фиксиран буфер и брояч. Постъпващите байтове се натрупват в буфера, а при наличие на цял ред (символ за нов ред) редът се извлича и остатъкът се премества в началото на буфера.

8.2 Функции

- `reader_init` — свързване с файлов дескриптор и буферът се изпразва.
- `reader_fill` — извършва се точно едно четене от сокета и получените байтове се добавят към буфера; връща се броят байтове, нула при край на потока или `-1` при грешка. Това е единствената функция, чрез която се осъществява достъп до мрежата.
- `reader_getline` — в наличните в буфера байтове се търси символ за нов ред. При намиране, редът се копира (без завършващите символи за нов ред), остатъкът се премества напред и се връща `1`, а при липса на цял ред се връща `0` без четене от мрежата. Чрез това разделяне между извличането от буфера и четенето от мрежата се осигурява работата на сървъра.
- `reader_getline_blocking` — комбинирана функция, при която последователно се опитва извличане на ред, а при липса се извършва четене, докато бъде получен цял ред. Тази функция се използва от клиента.
- `write_all` — при изпращане е възможно от едно извикване на `write` да бъде приета само част от данните, поради което записът се повтаря в цикъл, докато всички байтове бъдат изпратени, като прекъсванията от сигнал се обработват.
- `send_str` — обвивка, чрез която се изпраща низ, завършващ с нулев символ.

В резултат от това в `server.c` и `client.c` не се обработват частични четения и записи, нито се определят границите на редовете; вместо това се използват операциите за получаване и изпращане на цял ред.

9. Предоставена библиотека `game.c` и `game.h`

Игровата логика се съдържа в предоставената библиотека. В нея се обработва отделна дума, без да се използват мрежови механизми. От сървъра тази библиотека се прилага за всеки от двамата участници.

9.1 Множество от букви като битово поле

Чрез типа `letter_set_t` до 26 признака (по един за всяка буква) се пакетират в битовите на 32-битово цяло число, при което буквата „a“ съответства на бит 0, „b“ — на бит 1 и така нататък. Операциите се изпълняват чрез побитови действия — добавянето чрез

побитово „или“, премахването чрез побитово „и-не“, а проверката за принадлежност чрез побитово „и“. Този запис е ефективен за проверка дали дадена буква се съдържа в множеството, но при него редът на предположенията не се запазва. Това обстоятелство обуславя поддържането на отделен подреден списък с грешни букви на сървъра (раздел 10.6).

9.2 Структура `secret_word_t` и функции

- `secret_word_init_from_c_string` — заделя се памет, думата се нормализира до малки букви и се инициализира; при празна или невалидна дума се връща неуспех, поради което валидирането съвпада с прилаганото от сървъра.
- `secret_word_guess` — буквата се нормализира. При вече направено предположение се връща грешен отговор, а в противен случай всички съвпадащи позиции се разкриват и се връща вярно предположение, като при грешка буквата се добавя към множеството на грешните.
- `secret_word_letter_at` — указва се дали дадена позиция е разкрита и при разкрита позиция се връща буквата; чрез тази функция от сървъра се изгражда маската.
- `secret_word_is_solved` — връща се истина, когато всички позиции са разкрити.
- `secret_word_incorrect_guess_count` и `secret_word_free` — определя се броят грешки и се освобождава заделената памет.

10. Сървър `hangman-server`

10.1 Структура `player_t`

```
typedef struct player {
    int          fd;                // сокетът на участника
    line_reader_t reader;           // буфериран четец за сокета
    secret_word_t target;           // думата за отгатване от участника
    char         incorrect[27];     // грешни букви в реда на опитване
    int          incorrect_len;
    bool         solved;            // признак за отгатната дума
} player_t;
```

10.2 Стартиране и приемане на двама участници

В функцията `main` сигналът `SIGPIPE` се игнорира, създава се слушащ сокет, извършва се привързване към `0.0.0.0` и зададения порт, извежда се точно „Listening on <порт>...“ и буферът на изхода се изпразва, след което приемането на връзки се изпълнява в цикъл. За всяка нова връзка се прочита първият ред (подадената дума), който се валидира; при неуспех се изпраща „ERR invalid word“ и връзката се затваря, без да бъде отчетена. Само валидните заявки заемат място в масива с участници. След приемането на двама валидни участници слушащият сокет се затваря, с което се предотвратява присъединяването на трети клиент.

Получателят, използван при приемането, се запазва в структурата на участника. По този начин, ако от бърз клиент думата и първата буква са изпратени в един и същ пакет, вече буферираните байтове не се губят.

10.3 Валидиране на думата

Думата се счита за валидна само ако не е празна и всеки от символите ѝ е буква, което съответства на правилата в game.c. Валидирането се извършва преди началото на играта.

10.4 Изграждане на маската

```
for (i по дължината на думата) {
    if (secret_word_letter_at(w, i, &c) == REVEALED)
        out[i] = c;    // позицията е разкрита
    else
        out[i] = '_';  // позицията е скрита
}
```

За всяка позиция от game.c се установява дали е разкрита; разкритите букви се копират, а останалите се заменят със символ за подчертаване. По този начин тайната дума не напуска сървъра.

10.5 Обработка на предположение

От реда, изпратен от клиента, се извлича първият буквен символ и се подава на secret_word_guess, чрез което позициите се разкриват и се връща съответният признак. При грешно предположение буквата се добавя към подредения списък с грешки. След това чрез secret_word_is_solved се проверява дали думата е отгатната и се изпраща обновеното състояние. Повторно предположена буква не се отчита повторно.

10.6 Необходимост от отделен подреден списък с грешки

Тъй като в game.c грешките се съхраняват в битово поле, при което редът на предположенията не се запазва (раздел 9.1), а в изискванията, грешните букви се представят в реда в който са били въведени от участника, на сървъра се поддържа отделен подреден масив, в който всяка грешна буква се добавя при въвеждането ѝ. Същинската логика се изпълнява от битовото поле в game.c, докато подреденият масив служи единствено за запазване на реда при визуализацията.

10.7 Главен цикъл и обслужване на двамата участници

Тъй като двамата участници играят едновременно, и двата сокета трябва да бъдат наблюдавани и данните на всеки участник да бъдат обработени правилно и да бъдат върнати на правилния сокет. За постигането на това съществуват три подхода. Първия вариант е използването на нишки би било въведено споделено състояние, за чиято защита са необходими заключвания с цел избягване на условия, при които двете нишки се конкурират и си пречат една на друга. Втория вариант представлява последователни блокиращи четения, но при тях обслужването би било нарушено, тъй като при изчакване на единия участник постъпващите данни от другия биха били пренебрегнати.

Използван е трети подход — мултиплексиране на входно-изходните операции чрез `poll()`, при който в рамките на една нишка от операционната система се установява кой сокет разполага с готови данни и се обслужва именно той, без нишки, заключвания и състезателни условия.

```
poll(pfds, 2, -1); // изчакване, докато сокет стане четим
за всеки готов сокет:
    reader_fill(...) // едно четене
    while (reader_getline(...)) process_guess(...)
if (двата са отгатнали) { send_results(...); край }
```

На `poll` се подават дескрипторите и наблюдаваните събития, като стойност `-1` означава неограничено изчакване. След връщане за всеки готов сокет се извършва едно четене и се обработват всички налични цели редове, тъй като в едно четене могат да постъпят няколко предположения. Връщане на нула от четенето се интерпретира като разкачен участник и играта се прекратява коректно.

10.8 Определяне и изпращане на резултата

```
if (inc0 < inc1) { code0 = "WIN"; code1 = "LOSE"; }
else if (inc0 > inc1) { code0 = "LOSE"; code1 = "WIN"; }
else { code0 = "TIE"; code1 = "TIE"; }
```

За победител се определя участникът с по-малък брой грешки, а при равен брой се обявява равенство. На всеки участник се изпраща съобщение `RESULT` със съответния код, неговите грешни букви и тези на опонента, което е достатъчно за извеждането на трите изисквани реда.

11. Клиент `hangman-client`

Клиентът е реализиран опростено, тъй като цялата логика е съсредоточена в сървър. Последователността на действията е следната:

1. **Обработка на аргументите** — приемат се хост, порт и дума за опонента; при неправилен брой аргументи се извежда указание за употреба и програмата приключва с код 1.
2. **Преобразуване на адреса и свързване** — чрез `getaddrinfo` и цикъл с `connect` (раздел 6.2).
3. **Подаване на думата** — думата за опонента се изпраща незабавно като първи ред, който се прочита от сървър.
4. **Цикъл на отгатване** — съобщенията се получават последователно и се обработват според вида им: при `ERR` се извежда грешка и програмата приключва с код 1. При `STATE` състоянието се извежда и при наличие на скрита буква се прочита една буква от стандартния вход и се изпраща. При `RESULT` резултатът се извежда и програмата приключва.

5. **Изчакване на резултата** — след отгатване на собствената дума получаването продължава до постъпване на реда RESULT, тъй като опонентът може все още да играе, след което резултатът се извежда.

Преобразуването на компактния запис на грешните букви във вид с разделител запетая и интервал, както и прочитането на единичен символ от входа, се извършват чрез две помощни функции. По този начин натискането на интервал или нов ред не нарушава протокола, тъй като се изпраща само първата използвана буква. Състоянието „отгатнато“ се установява единствено по липсата на скрити позиции в маската, поради което истинската дума не е необходима на клиента.

12. Устойчивост и допълнителни детайли

12.1 Игнориране на SIGPIPE

При запис към сокет, чийто насрещен край е затворен, се доставя сигнал SIGPIPE, чието поведение по подразбиране е прекратяване на процеса. За да не бъде допуснато прекратяване на едната страна при разкачване на другата, в двете програми този сигнал се игнорира, поради което при такъв запис се връща грешка, която се обработва коректно.

12.2 Изпразване на изходния буфер

Изходът към канал (pipe) е блоково буфериран, поради което се задържа в буфер до запълването му. Тъй като при автоматичното тестване изходът се чете през канал, след всяко извеждане към потребителя буферът се изпразва, с което се осигурява незабавното му появяване.

12.3 Обработка на прекъсвания

Системните извиквания read, write, accept и poll могат да бъдат прекъснати от сигнал и да приключат преждевременно. В тези случаи извикванията се повтарят, вместо да бъдат третираны като действителна грешка.

12.4 Граници и безопасност на буферите

Дължината на думата е ограничена до 256 знака. Копирането се извършва с явно зададени размери, а масивът с грешни букви е ограничен до 26 елемента, поради което препълване не е възможно. Дължината на реда е ограничена от размера на буфера на четеща.

13. Процес на работа по проекта

Реализацията е извършена в следната последователност:

1. **Анализ на заданието** — открити са ключовите изисквания: приемане на точно двама клиенти, извеждане на „Listening on <порт>...“, защита срещу подсказване, формат на изхода и забрана за изменение на game.c.

2. **Проектиране на архитектурата** — възприет е моделът, при който ролята на съдия се изпълнява от сървъра, а клиентът е максимално опростен, и е обоснован изборът на TCP.
3. **Проектиране на протокола** — дефиниран е текстов протокол по редове с три съобщения от сървъра, като сигурността на комуникацията се осигурява чрез маскираната дума.
4. **Реализиране на помощната библиотека net-lib** — въз основа на свойството на TCP като байтов поток са разработени буфериран четец на редове и надеждно записване.
5. **Реализиране на сървъра** — приемане на двама участници, валидиране, кръстосано разпределение на думите, главен цикъл с poll() и отчитане на резултата.
6. **Реализиране на клиента** — свързване чрез getaddrinfo и цикъл на визуализиране, четене и изпращане с обработка на трите вида съобщения.
7. **Изготвяне на Makefile** — с първи target „all“, чрез който двата изпълними файла се компилират с опции -Wall -Wextra -O2 -std=c11.
8. **Тестване** — публичните тестове са преминати изцяло, като допълнително са проверени случаите на победа, загуба и равенство, невалидна дума, повтарящи се и повторно опитани букви.

14. Изграждане (Makefile)

```
CC      = gcc
CFLAGS  = -Wall -Wextra -O2 -std=c11
COMMON  = game.c net-lib.c
HEADERS  = game.h net-lib.h

all: hangman-server hangman-client

hangman-server: server.c $(COMMON) $(HEADERS)
$(CC) $(CFLAGS) -o hangman-server server.c $(COMMON)

hangman-client: client.c $(COMMON) $(HEADERS)
$(CC) $(CFLAGS) -o hangman-client client.c $(COMMON)

clean:
rm -f hangman-server hangman-client

.PHONY: all clean
```

Първият target е „all“, чрез който двата изпълними файла се компилират в директорията на проекта, както се очаква от заданието и от тестовата система. Всеки изпълним файл се свързва от съответния си файл с разширение .c заедно със споделените game.c и net-lib.c. Чрез опциите -Wall и -Wextra се активира широк набор предупреждения, които биха дали повече информация при грешки и бъгове, чрез -std=c11 се фиксира използваният стандарт, а чрез -O2 се прилага стандартна оптимизация.

15. Пускане и тестване

```
make                                # компилиране на двата файла

# терминал 1
./hangman-server 5555              # извежда: Listening on 5555...

# терминал 2 (трети аргумент: думата за опонента)
./hangman-client 127.0.0.1 5555 cat

# терминал 3
./hangman-client 127.0.0.1 5555 dog

# автоматични публични тестове
python3 test_hangman_public.py     # 6/6 успешни
```

16. Обобщение на решенията

Моделът клиент-сървър е избран, за да бъде осигурен единствен достоверен източник на данни и да се постигне по-високо ниво на сигурност, Протоколът TCP е предпочетен пред UDP заради подредбата на пакети и поддържането на връзка. Текстовият протокол по редове е избран заради улеснените тестване и отстраняване на грешки. Цялата игрова логика е съсредоточена в сървъра опростяване на клиента. Маскираната дума в съобщението STATE гарантира, че тайната дума не напуска сървъра.

Мултиплексирането чрез poll() е предпочетено пред нишки, за да бъдат обслужвани двамата участници едновременно без заключвания и условия в които двете клиентски програми се надпреварват за вниманието на сървъра. Буферираният четец в net-lib е необходим поради начина на работа на TCP като байтов поток, а надеждното записване гарантира пълната доставка на съобщенията. Отделният подреден списък с грешки компенсира загубата на ред в битовото поле на game.c. Игнорирането на SIGPIPE и изпразването на изходния буфер осигуряват устойчивост.

17. Структура на файловете

Проектът се състои от следните файлове. Сървърът се реализира в server.c, а клиентът — в client.c. Помощната библиотека се съдържа в net-lib.c и net-lib.h. Предоставената и игрова логика се намира в game.c и game.h. Изграждането се описва в Makefile, чийто първи target е „all“. Публичните автоматични тестове са разположени в test_hangman_public.py.